Hands-On Exercise 3: Web API using Custom Model Class, Authorization Filter and Exception Filter

Scenario

The objective of this exercise is to create a custom Employee model, return a list of employee objects through a Web API, implement a custom authorization filter to validate request headers, and implement a custom exception filter to handle runtime exceptions.

Objective

- Create a custom Employee model class.
- Return a list of Employee objects using a GET action method.
- Use the **FromBody** attribute to receive request data.
- Create a custom authorization filter.
- Create a custom exception filter.
- Test the API using Swagger.

Project Name

FirstWebAPI

Implementation

Step 1: Open Existing Project

Opened the existing **FirstWebAPI** project created in the previous labs.

Step 2: Install Required Package

Installed the compatibility package using NuGet Package Manager.

Install-Package Microsoft.AspNetCore.Mvc.WebApiCompatShim

Step 3: Create Model Classes

Created a **Models** folder and added the following classes:

- Employee.cs
- Department.cs
- Skill.cs

The **Employee** model contains:

- Id
- Name
- Salary
- Permanent

- Department
- Skills
- DateOfBirth

The **Department** model contains:

- Id
- Name

The **Skill** model contains:

- Id
- Name

Step 4: Modify EmployeeController

Created **EmployeeController** with Read and Write action methods.

A private method named **GetStandardEmployeeList()** was created to return a list of Employee objects.

The GET action method calls this private method and returns the employee list.

The POST action method accepts an Employee object using the **FromBody** attribute.

The PUT action method updates employee information.

The GET action method was decorated with:

```
[HttpGet]
[AllowAnonymous]
[ProducesResponseType(typeof(List<Employee>), StatusCodes.Status200OK)]
[ProducesResponseType(StatusCodes.Status500InternalServerError)]
```

Step 5: Create Custom Authorization Filter

Created a new folder named **Filters**.

Added a class named **CustomAuthFilter**.

The filter inherits from **ActionFilterAttribute**.

The **OnActionExecuting()** method was overridden to validate the request.

The filter performs the following validations:

- Checks whether the **Authorization** header exists.
- If the header is missing, returns:

Invalid request - No Auth token

- If the Authorization header does not contain **Bearer**, returns:

Invalid request - Token present but Bearer unavailable

The filter was applied to **EmployeeController**.

Step 6: Create Custom Exception Filter

Created a class named **CustomExceptionFilter**.

Implemented **IExceptionFilter**.

Overrode the **OnException()** method.

The filter performs the following actions:

- Captures exception details.
- Writes exception information into a log file.
- Returns **500 Internal Server Error** to the client.
- Prevents the application from crashing.

Step 7: Register Exception Filter

Registered **CustomExceptionFilter** in **Program.cs**.

The filter is injected using Dependency Injection.

Step 8: Build the Project

Built the application successfully using:

Build → Rebuild Solution

Step 9: Execute the Application

Executed the application using:

Ctrl + F5

Swagger UI opened successfully.

Step 10: Test GET Method

Executed the **GET /api/Employee** endpoint.

The API returned the list of Employee objects successfully.

Swagger displayed:

200 OK

Step 11: Test POST Method

Executed the **POST /api/Employee** endpoint.

The Employee object was sent in JSON format using the request body.

The **FromBody** attribute successfully mapped the JSON request to the Employee model.

Step 12: Test Authorization Filter

Executed the POST request without the Authorization header.

Response:

400 Bad Request

Invalid request - No Auth token

Executed the request with an invalid Authorization header.

Response:

400 Bad Request

Invalid request - Token present but Bearer unavailable

Executed the request using:

Authorization: Bearer <Token>

The request was processed successfully.

Step 13: Test Exception Filter

Triggered an exception from the GET action method.

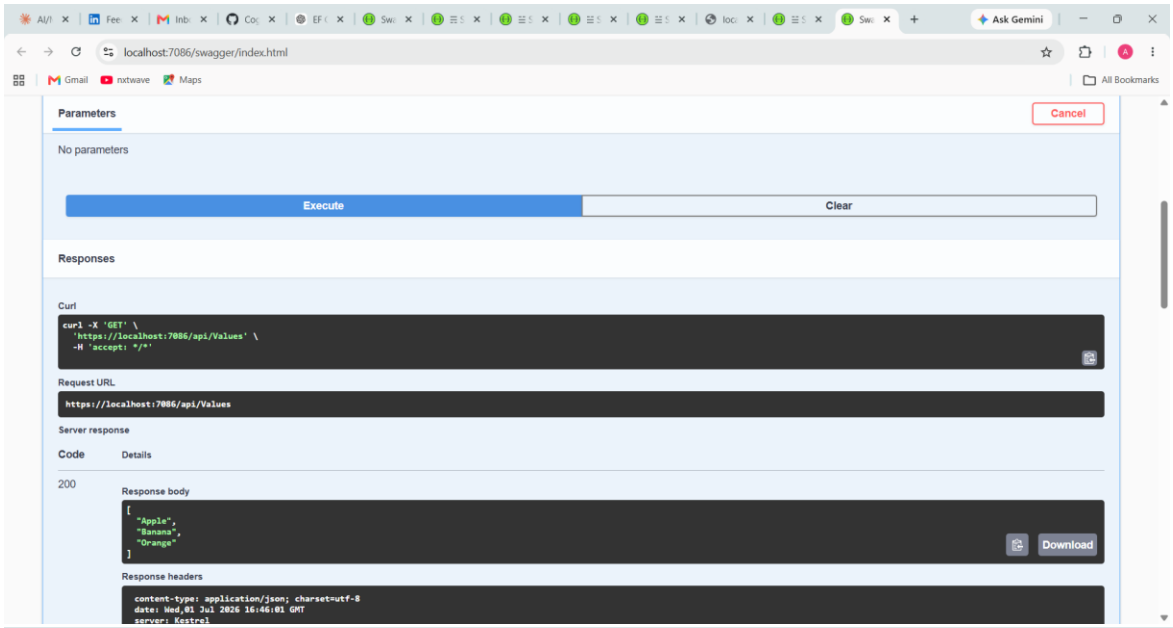
Swagger returned:

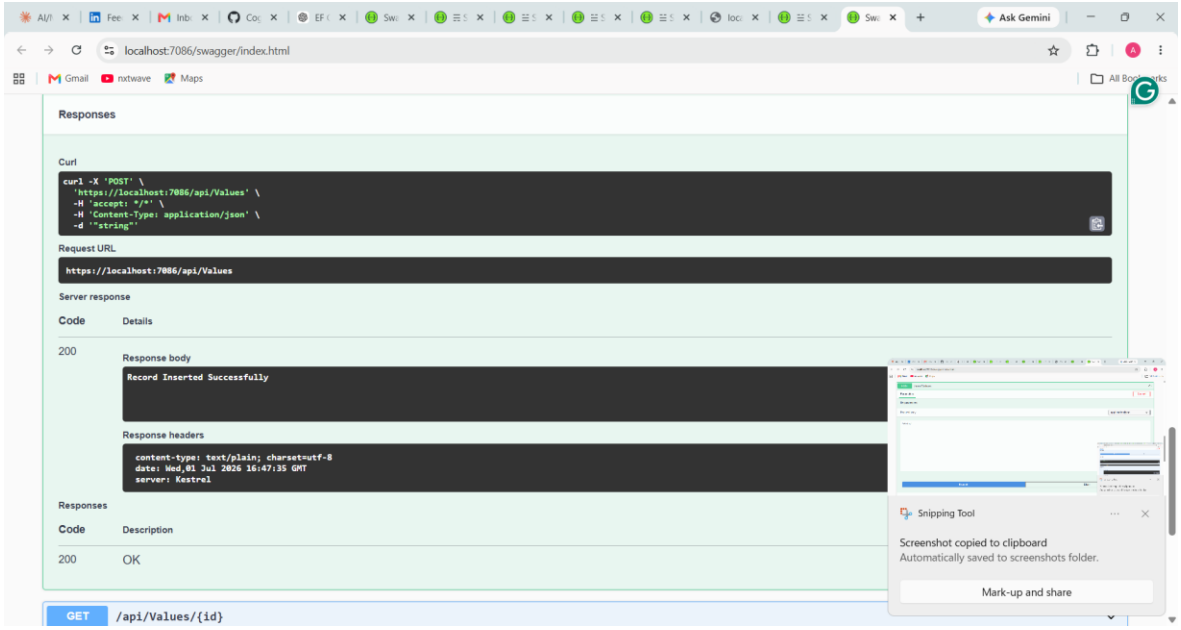
500 Internal Server Error

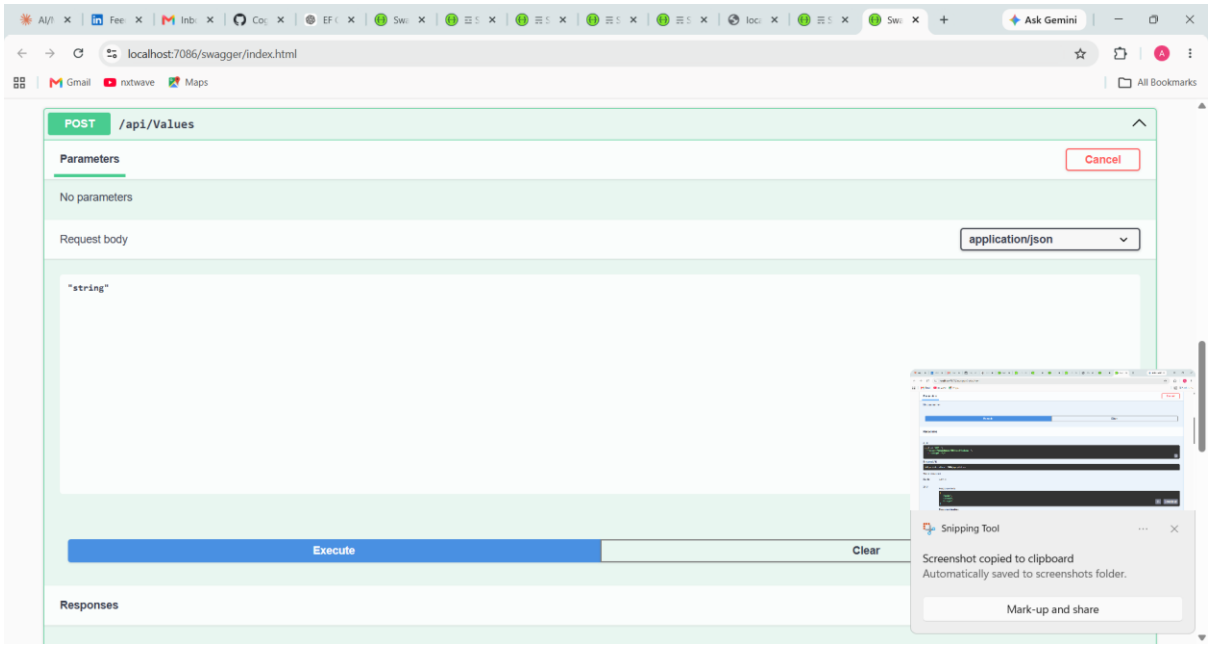
The exception details were written into the application log file.

Result

A custom **Employee** model was successfully created and exposed through the ASP.NET Core Web API. The GET action method returned a list of Employee objects, while the POST action method accepted JSON data using the **FromBody** attribute. A custom authorization filter validated incoming Authorization headers, and a custom exception filter handled runtime exceptions by logging the error and returning an appropriate HTTP 500 response. The application was successfully tested using Swagger.


localhost:7086/swagger/index.html
Parameters
Cancel
No parameters
Execute
Clear
Responses
Curl
curl -X 'GET' \
'https://localhost:7086/api/Values' \
-H 'accept: */*'
Request URL
https://localhost:7086/api/Values
Server response
Code
Details
200
Response body
[
"Apple",
"Banana",
"Orange"
]
Download
Response headers
content-type: application/json; charset=utf-8
date: Wed,01 Jul 2026 16:46:01 GMT
server: Kestrel


localhost:7086/swagger/index.html
Responses
Curl
curl -X 'POST' \
'https://localhost:7086/api/Values' \
-H 'accept: */*' \
-H 'Content-Type: application/json' \
-d '"string"'
Request URL
https://localhost:7086/api/Values
Server response
Code
Details
200
Response body
Record Inserted Successfully
Response headers
content-type: text/plain; charset=utf-8
date: Wed,01 Jul 2026 16:47:35 GMT
server: Kestrel
Responses
Code
Description
200
OK
GET
/api/Values/{id}
Snipping Tool
Screenshot copied to clipboard
Automatically saved to screenshots folder.
Mark-up and share


Ask Gemini
localhost:7086/swagger/index.html
Gmail
nxtwave
Maps
All Bookmarks
POST
/api/Values
Parameters
Cancel
No parameters
Request body
application/json
"string"
Execute
Clear
Responses
Snipping Tool
Screenshot copied to clipboard
Automatically saved to screenshots folder.
Mark-up and share